

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Laboratory Work 1 & 2:

The OS as a Resource Manager (Linux) & Meet xv6 (RISC-V)

Elaborated: st. gr. FAF-243

Șoimu Ionuț

Verified: asist. univ.

Reițman Patricia

Chișinău – 2026

Contents

1 Lab 1: Track A – The OS as a Resource Manager (Linux) 2
1.1 Before you start . . . 2
1.2 Part 1. Files and directories . . . 2
1.2.1 1. Navigate and Inspect . . . 2
1.2.2 2. Create, copy, rename, remove, and watch the tree change . . . 4
1.2.3 3. Look at ownership and permissions . . . 5
1.3 Part 2. Processes . . . 5
1.3.1 List the processes the OS is currently juggling . . . 5
1.3.2 Start a background process, find it, then stop it . . . 6
1.3.3 Peek at what the kernel exposes about a process through /proc . . . 6
1.4 Part 3. Memory . . . 8
1.4.1 Commands . . . 8
1.5 Part 4. Devices and Storage . . . 8
1.5.1 Commands . . . 8
1.6 Wrap up . . . 10

2 Lab 2: Track B – Meet the OS You Will Build (xv6) 11
2.1 Before you start . . . 11
2.2 Part 1. Build it and boot it . . . 11
2.2.1 Compile the kernel and the user programs, then launch xv6 in the emulator 11
2.3 Part 2. Use xv6 as the Unix it is . . . 12
2.4 Part 3. Read the source . . . 13
2.4.1 Read a user program . . . 13
2.4.2 System calls declaration in user.h and handlers in kernel/ . . . 15
2.5 Part 4. Write your first xv6 program . . . 16
2.5.1 Create user/sleep.c . . . 16
2.5.2 Added $U/_sleep in Makefile . . . 16
2.5.3 Rebuild, boot, and test your program inside xv6 . . . 17

3 Conclusion 18

1 Lab 1: Track A – The OS as a Resource Manager (Linux)

1.1 Before you start

Identification

```
➜  ~ mkdir -p ~/os-lab1 && cd ~/os-lab1
➜  os-lab1 whoami
ionut
➜  os-lab1 uname -a
Linux arch 7.2.8-1-cachyos #1 SMP PREEMPT_DYNAMIC Sat, 26 Sep 2026 19:43:45 +0000
    x86_64 GNU/Linux
➜  os-lab1 uptime
 16:11:53 up  2:34,  1 user,  load average: 0.58, 0.57, 0.62
➜  os-lab1
```

1.2 Part 1. Files and directories

1.2.1 1. Navigate and Inspect

```
➜  os-lab1 pwd
/home/ionut/os-lab1
➜  os-lab1 ls -la /
total 20K
dr-xr-xr-x   1 root root  142 Jun 18 15:01 .
dr-xr-xr-x   1 root root  142 Jun 18 15:01 ..
lrwxrwxrwx   1 root root    7 Oct 12  2025 bin -> usr/bin
drwx------   5 root root 4.0K Jan  1  1970 boot
drwxr-xr-x  20 root root 4.7K Sep 28 13:38 dev
drwxr-xr-x   1 root root 3.8K Sep 28 15:34 etc
drwxr-xr-x   1 root root   30 Sep 29  2025 home
lrwxrwxrwx   1 root root    7 Oct 12  2025 lib -> usr/lib
lrwxrwxrwx   1 root root    7 Oct 12  2025 lib64 -> usr/lib
drwxr-xr-x   1 root root   20 May  4 17:35 mnt
drwxr-xr-x   1 root root   40 Sep 19 00:25 opt
dr-xr-xr-x 335 root root    0 Sep 28 13:37 proc
drwx------   1 root root  108 Jul 18 17:31 root
drwxr-xr-x  35 root root  820 Sep 28 15:35 run
lrwxrwxrwx   1 root root    7 Oct 12  2025 sbin -> usr/bin
drwxr-x---   1 root root  130 Sep 28 14:13 .snapshots
drwxr-xr-x   1 root root   14 Sep 29  2025 srv
dr-xr-xr-x  13 root root    0 Sep 28 13:37 sys
drwxrwxrwt  19 root root  480 Sep 28 16:09 tmp
drwxr-xr-x   1 root root   80 Sep 28 14:13 usr
drwxr-xr-x   1 root root  126 Sep 28 13:37 var
➜  os-lab1 ls -la ~
total 1.3M
drwxr-x---+ 1 ionut ionut 1.7K Sep 28 16:13  .
drwxr-xr-x  1 root  root    30 Sep 29  2025  ..
drwxr-x---  1 ionut ionut   84 Aug  3 19:10  .android
drwxr-xr-x  1 ionut ionut  204 Jan  8  2026  .barony
drwxr-xr-x+ 1 ionut ionut 2.7K Sep 28 14:13  .cache
drwxr-xr-x  1 ionut ionut  112 Sep 14 16:46  .cargo
drwxr-x---+ 1 ionut ionut 2.9K Sep 25 16:32  .config
drwx------  1 ionut ionut   36 Sep 15 14:07  .copilot
drwx------  1 ionut ionut   18 Jun  8 11:20  .cups
```

```
drwxr-xr-x  1 ionut ionut   70 Oct  3  2025  .dbclient
drwxr-xr-x  1 ionut ionut  600 Aug 18 16:30  Desktop
drwxr-xr-x  1 ionut ionut  264 Aug  4 17:28  Documents
drwxr-xr-x  1 ionut ionut  194 Mar 19  2026  .dotnet
drwxr-xr-x  1 ionut ionut 3.2K Sep 28 15:59  Downloads
drwxr-xr-x  1 ionut ionut   12 Apr 27 12:08  .electron-gyp
drwxr-xr-x  1 ionut ionut  948 Sep 18 22:39  Games
drwxr-xr-x  1 ionut ionut   52 Sep 21 17:24  .gemini
drwx------  1 ionut ionut  116 Sep 29  2025  .gnupg
drwxr-xr-x  1 ionut ionut    6 Jan 26  2026  go
drwxr-xr-x  1 ionut ionut  140 Aug  3 19:10  .gradle
drwxr-xr-x  1 ionut ionut   50 Jun  8 11:35  .hplip
drwxr-xr-x  1 ionut ionut   30 Oct  2  2025  .java
drwxr-xr-x  1 ionut ionut  152 Dec  1  2025  .jdks
drwxr-xr-x  1 ionut ionut    8 Mar 15  2026  Library
drwx------  1 ionut ionut  228 Sep 22 13:28  .librewolf
drwxr-xr-x+ 1 ionut ionut   46 Jan 30  2026  .local
drwxr-xr-x  1 ionut ionut 1.7K Feb 13  2024  mama
drwxr-xr-x  1 ionut ionut   16 Oct 17  2025  .masterpdfeditor
drwx------  1 ionut ionut   34 Sep 29  2025  .mozilla
drwxr-xr-x  1 ionut ionut    0 Sep 29  2025  Music
drwxr-xr-x  1 ionut ionut   26 Jun 30 20:39 'My Games'
drwxr-xr-x  1 ionut ionut    0 Dec 28  2025  .netscanner
drwxr-xr-x  1 ionut ionut   92 Feb  2  2026  .npm
drwxr-xr-x  1 ionut ionut   26 Mar 19  2026  .nuget
drwxr-xr-x  1 ionut ionut  356 Sep 22 12:18  .oh-my-zsh
drwx--x---+ 1 ionut ionut   32 Oct  1  2025  old.desktop
drwxr-xr-x  1 ionut ionut    0 Sep 28 16:11  os-lab1
drwxr-xr-x  1 ionut ionut  164 Nov 16  2025  .paradoxlauncher
drwxr-xr-x+ 1 ionut ionut  562 May  4 17:38  Pictures
drwx------  1 ionut ionut   10 Sep 29  2025  .pki
drwxr-xr-x  1 ionut ionut  194 Sep 28 16:09  Programming
drwxr-xr-x  1 ionut ionut    0 Apr 14 17:17  Projects
drwxr-xr-x  1 ionut ionut  118 Jul 27 17:13  pt
drwxr-xr-x  1 ionut ionut    0 Sep 29  2025  Public
drwxr-xr-x  1 ionut ionut   96 Jun 29 17:48  qsstv
drwxr-xr-x  1 ionut ionut   92 Sep 22 16:55  Recordings
drwxr-xr-x  1 ionut ionut  168 May 15 10:20  .ServiceHub
drwxr-xr-x  1 ionut ionut   78 Oct 21  2025  sketchbook
drwx------  1 ionut ionut  126 Jun 21 14:42  .ssh
drwxr-xr-x  1 ionut ionut  190 Sep 21 15:10  .steam
drwxr-xr-x  1 ionut ionut  112 Aug 18 16:30  temp
drwxr-xr-x  1 ionut ionut    0 Sep 29  2025  Templates
drwxr-xr-x  1 ionut ionut   22 Mar 15  2026  UnrealEngine
drwxr-xr-x  1 ionut ionut    6 Sep 29  2025  .var
drwxr-xr-x  1 ionut ionut  144 Jun 21 18:09  .VASSAL
drwxr-xr-x  1 ionut ionut  100 Feb 14  2026  Videos
drwxr-xr-x  1 ionut ionut   44 Sep 29  2025  .vscode
drwxr-xr-x  1 ionut ionut   26 May  6 21:11  .vscode-shared
drwxr-xr-x  1 ionut ionut  154 Sep 18 23:29  .wine
drwxr-xr-x  1 ionut ionut   10 Feb  2  2026  .yarn
drwxr-xr-x  1 ionut ionut  412 Jul 14 17:37  .ZAP
drwxr-xr-x  1 ionut ionut   12 Mar 15  2026  .zen
drwxr-xr-x  1 ionut ionut  400 Aug  8 18:56  Zomboid
drwx------  1 ionut ionut   54 Jan 13  2026  .zoom
drwx------  1 ionut ionut   12 Feb 17  2026  .zotero
drwxr-xr-x  1 ionut ionut  128 May 15 11:35  Zotero
-rw-r--r--  1 ionut ionut    0 Mar 23  2026  a.out
```

```
-rw-------  1 ionut ionut  13K Oct 16  2025  .bash_history
-rw-r--r--  1 ionut ionut   21 Aug  1  2025  .bash_logout
-rw-r--r--  1 ionut ionut   22 Jan 27  2026  .bash_profile
-rw-r--r--  1 ionut ionut  172 Jan 27  2026  .bashrc
-rw-r--r--  1 ionut ionut  267 Mar 21  2026  .gitconfig
-rw-r--r--  1 ionut ionut  345 Sep 28 13:39  .gtkrc-2.0
-rw-------  1 ionut ionut 346K Sep 28 16:13  .histfile
-rw-r--r--  1 ionut ionut 1.1K Aug 18 12:37  .jc141rc
-rw-r--r--  1 ionut ionut  243 Sep 27 15:15  .p7zip
-rw-r--r--  1 ionut ionut 4.3K Jul 27 22:59  .packettracer
-rw-------  1 ionut ionut  256 Feb 13  2026  .pulse-cookie
lrwxrwxrwx  1 ionut ionut   30 Sep 21 12:31  .steampath -> /home/ionut/.steam/sdk32/
    steam
lrwxrwxrwx  1 ionut ionut   28 Sep 21 12:31  .steampid -> /home/ionut/.steam/steam.
    pid
-rw-r--r--  1 ionut ionut 110K Jul 26 18:22  temp2.txt
-rwxr-xr-x  1 ionut ionut 1.3K Jun 12 21:42  temp.sh
-rw-r--r--  1 ionut ionut 176K Jul 14 17:50  Untitled.csv
-rw-------  1 ionut ionut 6.5K Jun  9 13:38  .viminfo
-rw-r--r--  1 ionut ionut  410 Sep 27 13:06  .wget-hsts
-rw-r--r--  1 ionut ionut   47 Mar 27  2026  .Xresources
-rw-r--r--  1 ionut ionut   40 Mar 27  2026  .Xresources.backup1774635780
-rw-r--r--  1 ionut ionut  55K Sep 27 15:40  .zcompdump
-rw-r--r--  1 ionut ionut  53K May 22 11:44  .zcompdump-arch-5.9
-rw-r--r--  1 ionut ionut  56K Jul  5 12:42  .zcompdump-arch-5.9.1
-r--r--r--  1 ionut ionut 120K Jul  5 12:42  .zcompdump-arch-5.9.1.zwc
-rw-r--r--  1 ionut ionut  56K Sep 27 15:40  .zcompdump-arch-5.9.2
-r--r--r--  1 ionut ionut 121K Sep 27 15:40  .zcompdump-arch-5.9.2.zwc
-r--r--r--  1 ionut ionut 124K May 22 11:44  .zcompdump-arch-5.9.zwc
-rw-r--r--  1 ionut ionut    0 Aug 17 13:37  .zshenv
-rw-r--r--  1 ionut ionut 1.6K Oct 16  2025  .zshrc
-rw-r--r--  1 ionut ionut  997 Oct 16  2025  .zshrc.zni
➜  os-lab1 cd /etc && ls | head
total 1.1M
drwxr-xr-x 1 root root  3.8K Sep 28 15:34 .
dr-xr-xr-x 1 root root   142 Jun 18 15:01 ..
drwxr-xr-x 1 root root    32 Nov  4  2025 acpi
drwxr-xr-x 1 root root    12 Sep 29  2025 alsa
drwxr-xr-x 1 root root   122 Sep 14 16:36 ananicy.d
drwxr-xr-x 1 root root    10 Apr 27 12:11 apparmor.d
drwxr-xr-x 1 root root     0 Nov  1  2025 audisp
drwxr-xr-x 1 root root    86 Aug  3 13:04 audit
drwxr-xr-x 1 root root   142 Jun 28 20:08 avahi
➜  /etc
```

1.2.2 2. Create, copy, rename, remove, and watch the tree change

```
➜  /etc cd ~/os-lab1
➜  os-lab1 mkdir demo && cd demo
➜  demo echo "hello operating systems" > note.txt
➜  demo cp note.txt copy.txt
➜  demo mv copy.txt renamed.txt
➜  demo ls -l
total 8.0K
drwxr-xr-x 1 ionut ionut 38 Sep 28 16:20 .
drwxr-xr-x 1 ionut ionut  8 Sep 28 16:19 ..
-rw-r--r-- 1 ionut ionut 24 Sep 28 16:19 note.txt
```

```
-rw-r--r-- 1 ionut ionut 24 Sep 28 16:20 renamed.txt
➜  demo rm renamed.txt
➜  demo
```

1.2.3 3. Look at ownership and permissions

```
➜  demo ls -l note.txt
-rw-r--r-- 1 ionut ionut 24 Sep 28 16:19 note.txt
➜  demo chmod 600 note.txt
➜  demo ls -l note.txt
-rw------- 1 ionut ionut 24 Sep 28 16:19 note.txt
➜  demo chmod 644 note.txt
➜  demo
```

Observe:

1. **Who owns the files you just created, and which group?**
   The files that I just created belong to the owner user and the owner group, specifically `ionut` in this case.

2. **What do the ten characters at the start of `ls -l` mean for note.txt after chmod 600?**
   The first character shows the file type, in this case a regular file. The next 3 pairs of 3 characters shows the permission for the user, group and others, in this case only the user can read and write the file.

3. **Name two directories directly under / and say, in one line each, what they hold.**
   - `/bin/` – essential command binaries available to all users
   - `/home/` – users' home directories, containing saved files, personal settings, etc.

1.3 Part 2. Processes

1.3.1 List the processes the OS is currently juggling

```
➜  demo ps aux | head
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  22140 16716 ?        Ss   13:37   0:04 /usr/lib/systemd/
    systemd --switched-root --system --deserialize=59
root           2  0.0  0.0      0     0 ?        S    13:37   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    13:37   0:00 [
    pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   13:37   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   13:37   0:00 [kworker/R-sync_wq
    ]
root           6  0.0  0.0      0     0 ?        I<   13:37   0:00 [kworker/R-
    kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   13:37   0:00 [kworker/R-
    slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   13:37   0:00 [kworker/R-netns]
root          11  0.0  0.0      0     0 ?        I<   13:37   0:00 [kworker/0:0H-
    kblockd]
➜  demo ps aux | wc -l
332
```

```
➜  demo
```

1.3.2 Start a background process, find it, then stop it

```
➜  demo sleep 300 &
[1] 15696
➜  demo jobs
[1]  + running    sleep 300
➜  demo ps -ef | grep sleep
ionut      15696   12709  0 16:44 pts/2    00:00:00 sleep 300
ionut      15794   12709  0 16:45 pts/2    00:00:00 grep --color=auto sleep
➜  demo kill 15696
[1]  + 15696 terminated  sleep 300
➜  demo
```

1.3.3 Peek at what the kernel exposes about a process through /proc

```
➜  demo sleep 300 &
[1] 15976
➜  demo PID=$!
➜  demo ls /proc/$PID/
total 0
dr-xr-xr-x   9 ionut ionut 0 Sep 28 16:49 .
dr-xr-xr-x 332 root  root  0 Sep 28 13:37 ..
dr-xr-xr-x   2 ionut ionut 0 Sep 28 16:49 attr
lrwxrwxrwx   1 ionut ionut 0 Sep 28 16:49 cwd -> /home/ionut/os-lab1/demo
dr-x------   2 ionut ionut 3 Sep 28 16:49 fd
dr-xr-xr-x   2 ionut ionut 0 Sep 28 16:49 fdinfo
dr-x------   2 ionut ionut 0 Sep 28 16:49 map_files
dr-xr-xr-x   5 ionut ionut 0 Sep 28 16:49 net
dr-x--x--x   2 ionut ionut 0 Sep 28 16:49 ns
lrwxrwxrwx   1 ionut ionut 0 Sep 28 16:49 root -> /
dr-xr-xr-x   3 ionut ionut 0 Sep 28 16:49 task
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 arch_status
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 autogroup
-r--------   1 ionut ionut 0 Sep 28 16:49 auxv
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 cgroup
--w-------   1 ionut ionut 0 Sep 28 16:49 clear_refs
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 cmdline
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 comm
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 coredump_filter
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 cpu_resctrl_groups
-r--------   1 ionut ionut 0 Sep 28 16:49 environ
lrwxrwxrwx   1 ionut ionut 0 Sep 28 16:49 exe -> /usr/bin/sleep
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 gid_map
-r--------   1 ionut ionut 0 Sep 28 16:49 io
-r--------   1 ionut ionut 0 Sep 28 16:49 ksm_merging_pages
-r--------   1 ionut ionut 0 Sep 28 16:49 ksm_stat
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 limits
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 loginuid
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 maps
-rw-------   1 ionut ionut 0 Sep 28 16:49 mem
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 mountinfo
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 mounts
-r--------   1 ionut ionut 0 Sep 28 16:49 mountstats
```

```
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 numa_maps
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 oom_adj
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 oom_score
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 oom_score_adj
-r--------   1 ionut ionut 0 Sep 28 16:49 pagemap
-r--------   1 ionut ionut 0 Sep 28 16:49 personality
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 projid_map
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 sched
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 schedstat
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 sessionid
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 setgroups
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 smaps
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 smaps_rollup
-r--------   1 ionut ionut 0 Sep 28 16:49 stack
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 stat
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 statm
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 status
-r--------   1 ionut ionut 0 Sep 28 16:49 syscall
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 timens_offsets
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 timers
-rw-rw-rw-   1 ionut ionut 0 Sep 28 16:49 timerslack_ns
-rw-r--r--   1 ionut ionut 0 Sep 28 16:49 uid_map
-r--r--r--   1 ionut ionut 0 Sep 28 16:49 wchan
➜  demo cat /proc/$PID/status | head -20
Name:   sleep
Umask:  0022
State:  S (sleeping)
Tgid:   15976
Ngid:   0
Pid:    15976
PPid:   12709
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 64
Groups: 50 150 949 951 952 954 985 998 1000
NStgid: 15976
NSpid:  15976
NSpgid: 15976
NSsid:  12709
Kthread:        0
VmPeak:    6316 kB
VmSize:    6316 kB
VmLck:        0 kB
➜  demo kill $PID
[1]  + 15976 terminated  sleep 300
➜  demo
```

Observe:

1. **What is the PID of process number 1, and what is it (look it up)?**

   The PID for process 1 is, surprisingly, 1, it is systemd, the init system and service manager for Linux and so it is the very first user-space process started when the kernel is booted up, every other process is a descendant of process 1.

2. **Roughly how many processes were running on your idle VM?**

Roughly 323 processes, since this is not a vm, a large portion of these are internal kernel threads (such as [kthreadd], [kworker/...], and interrupt handlers) that assist the OS in managing hardware, alongside systemd background daemons and desktop environment tasks.

3. **From /proc/¡pid¿/status, what does the ”State:” line say for your sleeping process?**
   It says `S (sleeping)` indicating the process is idle and waiting an OS timer event.

1.4 Part 3. Memory

1.4.1 Commands

```
➜  demo free -h
               total        used        free      shared  buff/cache   available
Mem:            12Gi       4.7Gi       1.9Gi       192Mi       6.7Gi       8.1Gi
Swap:          4.0Gi       484Ki       4.0Gi
➜  demo cat /proc/meminfo | head -6
MemTotal:       13326576 kB
MemFree:         1964324 kB
MemAvailable:    8439400 kB
Buffers:            2744 kB
Cached:          6919884 kB
SwapCached:            0 kB
➜  demo sleep 300 & PID=$!
[1] 16929
➜  demo grep VmRSS /proc/$PID/status
VmRSS:     2632 kB
➜  demo kill $PID
[1]  + 16929 terminated  sleep 300
➜  demo
```

Observe:

1. **How much total RAM does the VM have, and how much is free?**
   The baremetal host has `13326576 kB` total RAM and `1964324 kB` technically free, while available is `8439400 kB`, which is the free, usable RAM in the sense of windows.

2. **What is swap, and how much is configured?**
   Swap is an overflow area (either a dedicated disk partition, file, or compressed RAM segment) utilized by the OS virtual memory manager to page out inactive memory pages, freeing physical RAM for active working sets and caching. Currently 4GB is allocated/configured.

3. **How much resident memory (VmRSS) does a bare ‘sleep‘ process use, and does that surprise you?**
   The bare sleep process uses `2632 kB` resident memory, which is technically a lot, but in Linux the resident memory accounts not only for the minimal code of the sleep binary itself, but also for mapping shared C libraries (libc.so), the runtime dynamic linker (ld-linux.so), the stack, and kernel page table structures allocated to manage the process’s virtual memory space.

1.5 Part 4. Devices and Storage

1.5.1 Commands

```
➜  demo df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme1n1p2  931G  834G   93G  90% /
devtmpfs        6.4G     0  6.4G   0% /dev
tmpfs           6.4G     0  6.4G   0% /dev/shm
efivarfs        246K   94K  147K  39% /sys/firmware/efi/efivars
tmpfs           2.6G  2.1M  2.6G   1% /run
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           6.4G   52K  6.4G   1% /tmp
/dev/nvme1n1p2  931G  834G   93G  90% /home
/dev/nvme1n1p2  931G  834G   93G  90% /var/cache/pacman/pkg
/dev/nvme1n1p2  931G  834G   93G  90% /var/log
/dev/nvme1n1p1 1022M  110M  913M  11% /boot
tmpfs           1.3G   44M  1.3G   4% /run/user/1000
➜  demo lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
zram0       253:0    0     4G  0 disk [SWAP]
nvme0n1     259:0    0 476.9G  0 disk
|-nvme0n1p1 259:1    0   100M  0 part
|-nvme0n1p2 259:2    0    16M  0 part
|-nvme0n1p3 259:3    0  96.8G  0 part
|-nvme0n1p4 259:4    0   736M  0 part
`-nvme0n1p5 259:5    0 379.3G  0 part
nvme1n1     259:6    0 931.5G  0 disk
|-nvme1n1p1 259:7    0     1G  0 part /boot
`-nvme1n1p2 259:8    0 930.5G  0 part /var/log
                                      /var/cache/pacman/pkg
                                      /home
                                      /
➜  demo du -sh ~/os-lab1
4.0K    /home/ionut/os-lab1
➜  demo ls -l /dev | head
total 0
drwxr-xr-x  20 root  root         4.7K Sep 28 17:03 .
dr-xr-xr-x   1 root  root          142 Jun 18 15:01 ..
drwxr-xr-x   2 root  root          240 Sep 28 13:37 block
drwxr-xr-x   3 root  root           60 Sep 28 13:37 bus
drwxr-xr-x   2 root  root         5.2K Sep 28 17:03 char
drwxr-xr-x  14 root  root          280 Sep 28 13:37 cpu
drwxr-xr-x  10 root  root          200 Sep 28 13:37 disk
drwxr-xr-x   2 root  root           60 Sep 28 13:37 dma_heap
drwxr-xr-x   3 root  root          100 Sep 28 13:38 dri
➜  demo mount | head
/dev/nvme1n1p2 on / type btrfs (rw,relatime,compress=zstd:3,ssd,discard=async,
    space_cache=v2,subvolid=256,subvol=/@)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=6619192k,nr_inodes=1654798,mode=755,
    inode64,huge=advise)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,huge=advise,usrquota)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode
    =000)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,
    memory_recursiveprot,memory_hugetlb_accounting)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
efivarfs on /sys/firmware/efi/efivars type efivarfs (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
```

```
➜ demo
```

Observe:

1. **Which device is your root filesystem "/" mounted on?**
   The device my root filesystem is mounted on is `/dev/nvme1n1p2`.

2. **Give one entry from /dev and say what real thing it stands for.**
   `/dev/nvme1n1p2` represents the physical NVMe solid-state drive (a block storage hardware device).

3. **In one sentence: what does "everything is a file" mean, based on what you saw?**
   "Everything is a file" means that the operating system abstracts a lot of things like the hardware devices, the processes and kernel runtime state through a single unified filesystem namespace.

1.6 Wrap up

The operating system acts as the foundational resource manager of a computer by coordinating four core hardware resources: files and storage, processes, physical memory, and I/O devices. We observed file management through commands like `ls -l` and `chmod`, which revealed how the OS translates raw storage into a structured tree governed by access permissions, while process management was demonstrated using `ps` and `/proc/<pid>/status` as the kernel created, isolated, scheduled, and tracked running tasks. Finally, memory management was made visible with `free -h` and `meminfo` as the kernel balanced physical RAM allocations against dynamic caches and swap, while `lsblk` and `/dev` showed device management in action by exposing disparate hardware components through uniform file-like interfaces.

2 Lab 2: Track B – Meet the OS You Will Build (xv6)

2.1 Before you start

```
➜  demo git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
Cloning into '/home/ionut/xv6-riscv'...
remote: Enumerating objects: 8334, done.
remote: Counting objects: 100% (33/33), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 8334 (delta 19), reused 11 (delta 11), pack-reused 8301 (from 2)
Receiving objects: 100% (8334/8334), 17.67 MiB | 294.00 KiB/s, done.
Resolving deltas: 100% (4583/4583), done.
➜  demo cd ~/xv6-riscv
➜  xv6-riscv git:(riscv) ls
total 44K
drwxr-xr-x  1 ionut ionut  252 Sep 28 17:40 .
drwxr-x---+ 1 ionut ionut 1.7K Sep 28 17:45 ..
drwxr-xr-x  1 ionut ionut  122 Sep 28 17:40 .git
drwxr-xr-x  1 ionut ionut   18 Sep 28 17:40 .github
drwxr-xr-x  1 ionut ionut  664 Sep 28 17:40 kernel
drwxr-xr-x  1 ionut ionut   12 Sep 28 17:40 mkfs
drwxr-xr-x  1 ionut ionut  360 Sep 28 17:40 user
-rw-r--r--  1 ionut ionut  238 Sep 28 17:40 .clang-format
-rw-r--r--  1 ionut ionut   86 Sep 28 17:40 .dir-locals.el
-rw-r--r--  1 ionut ionut  248 Sep 28 17:40 .editorconfig
-rw-r--r--  1 ionut ionut  171 Sep 28 17:40 .gdbinit.tmpl-riscv
-rw-r--r--  1 ionut ionut   85 Sep 28 17:40 .gitignore
-rw-r--r--  1 ionut ionut 1.2K Sep 28 17:40 LICENSE
-rw-r--r--  1 ionut ionut 5.7K Sep 28 17:40 Makefile
-rw-r--r--  1 ionut ionut 2.4K Sep 28 17:40 README
-rwxr-xr-x  1 ionut ionut 6.1K Sep 28 17:40 test-xv6.py
➜  xv6-riscv git:(riscv)
```

2.2 Part 1. Build it and boot it

2.2.1 Compile the kernel and the user programs, then launch xv6 in the emulator

```
➜  xv6-riscv git:(riscv) make qemu
/bin/sh: line 1: bc: command not found
/bin/sh: line 1: [: : integer expected
riscv64-linux-gnu-gcc -march=rv64gc -g -ffile-prefix-map=/home/ionut/xv6-riscv=. -c -
    o kernel/entry.o kernel/entry.S
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-
    pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/ionut/xv6-riscv=. -march=rv64gc -
    std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -I. -fno-stack-
    protector -fno-pie -no-pie   -c -o kernel/start.o kernel/start.c
...
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init
    user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/
    _usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/
    _dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
    total 2000
balloc: first 962 blocks have been allocated
balloc: write bitmap block at sector 46
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
    nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,
    format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ QEMU: Terminated
➜  xv6-riscv git:(riscv)
```

2.3 Part 2. Use xv6 as the Unix it is

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35864
echo           2 4 34760
forktest       2 5 17704
grep           2 6 43288
init           2 7 35112
kill           2 8 34664
ln             2 9 34456
ls             2 10 41944
mkdir          2 11 34712
rm             2 12 34696
sh             2 13 57072
stressfs       2 14 35536
usertests      2 15 201520
grind          2 16 50336
wc             2 17 36832
zombie         2 18 34008
logstress      2 19 36680
forphan        2 20 35472
dorphan        2 21 34936
sync           2 22 34112
console        3 23 0
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
...
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 35864
echo           2 4 34760
wc             2 17 36832
sync           2 22 34112
console        3 23 0
$ wc README
48 336 2441 README
$ usertests -q
usertests starting
test copyin: OK
```

```
test copyout: OK
test copyinstr1: OK
...
test kernmem: usertrap(): unexpected scause 0xd pid=6496
OK
test sbrkfail: OK
test sbrkarg: OK
test validatetest: OK
test stacktest: OK
test nowrite: OK
test lazy_alloc: OK
...
ALL TESTS PASSED
$
```

Observe:

1. **List three programs xv6 ships with.**
   - `sh` – The command shell
   - `ls` – Lists directory contents
   - `cat` – Reads and prints file contents
2. **You just used a pipe (—) inside xv6. Which two OS features must exist for a pipe to work?**
   - **Process Creation / Concurrency (fork):** The OS must be able to create and run multiple processes simultaneously so the reader and writer can execute at the same time.
   - **Inter-Process Communication (pipe):** A mechanism (usually an in-memory buffer accessed via file descriptors) to pass data between those isolated processes.
3. **In one sentence: how does the xv6 shell compare to the Linux shell from Lab 1?**
   The xv6 shell is a stripped-down, educational implementation supporting basic commands, pipes, and redirection, but lacking the advanced scripting, history, and feature-richness of a standard Linux shell.

2.4 Part 3. Read the source

2.4.1 Read a user program

```
➜  xv6-riscv git:(riscv) sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
```

```
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
➜  xv6-riscv git:(riscv) sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
```

```
➜  xv6-riscv git:(riscv)
```

2.4.2 System calls declaration in user.h and handlers in kernel/

```
➜  xv6-riscv git:(riscv) cat user/user.h
#define SBRK_ERROR ((char *)-1)

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);
➜  xv6-riscv git:(riscv) grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
```

```
83:sys_write(void)
➜  xv6-riscv git:(riscv)
```

Observe:

1. **Which system calls does user/cat.c use, and what does each one ask the kernel to do?**
   - **read:** Asks the kernel to read up to a specified number of bytes from an open file descriptor (fd) into a user-space buffer (buf).
   - **write:** Asks the kernel to write a specified number of bytes (n) from a buffer (buf) to an open file descriptor (here, standard output fd = 1).
   - **open:** Asks the kernel to find/open the file at the specified pathname with the given access mode (`O_RDONLY`) and allocate/return a new file descriptor referring to it.
   - **close:** Asks the kernel to close the specified file descriptor, decrementing the file reference count and releasing file table entries/resources.
   - **exit:** Asks the kernel to terminate the calling process, clean up its address space and resources, and deliver the exit status code to the parent process.
2. **In which kernel file, and on which line, is sys_read implemented?**
   `sys_read` is implemented in the `kernel/sysfile.c` file on line 68 (lines 68–69).
3. **In one sentence: what is the difference between the code in kernel/ and the code in user/?**
   Code in `kernel/` runs in privileged supervisor mode with direct access to hardware, physical memory, and system data structures to manage resources, whereas code in `user/` runs in unprivileged user mode within isolated virtual address spaces and must invoke system calls to request services from the kernel.

2.5 Part 4. Write your first xv6 program

2.5.1 Create user/sleep.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));    // system call into the kernel
  exit(0);
}
```

2.5.2 Added $U/_sleep in Makefile

Added `$U/_sleep\` to the `UPROGS` list in `Makefile`.

2.5.3 Rebuild, boot, and test your program inside xv6

```
➜  xv6-riscv git:(riscv) ✗ make qemu
/bin/sh: line 1: bc: command not found
/bin/sh: line 1: [: : integer expected
riscv64-linux-gnu-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-
    pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/ionut/xv6-riscv=. -march=rv64gc -
    std=gnu99 -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -I. -fno-stack-
    protector -fno-pie -no-pie   -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_sleep user/sleep.
    o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_sleep > user/sleep.asm
riscv64-linux-gnu-objdump -t user/_sleep | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d'
    > user/sleep.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_sleep user/_forktest user/_grep
    user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/
    _stressfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/
    _forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953
    total 2000
balloc: first 997 blocks have been allocated
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
    nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,
    format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

Note

In this task I called the system call `sys_pause` not `sys_sleep` because in the latest version of xv6 the system call got renamed. This change was made in August 2025.

3 Conclusion

Across these two laboratory works, I explored operating system design and resource management from two complementary perspectives: first, observing and interacting with a modern production kernel (Linux) from user space, and second, examining and extending a minimal UNIX kernel (xv6 on RISC-V) from the inside.

In **Lab 1**, I investigated how the Linux kernel abstracts and multiplexes hardware resources. Through the `proc` and `sys` pseudo-filesystems, I inspected how the kernel maintains process state, memory layouts, and scheduling metrics without requiring specialized system utilities. Using command-line tools and shell features, I analyzed process lifecycles, signal dispatching (`SIGSTOP`, `SIGCONT`, `SIGKILL`), standard stream redirection, and inter-process communication pipelines. Additionally, I explored storage organization, observing how inodes, hard links, symbolic links, and file descriptors structure persistent and temporary data on disk.

In **Lab 2**, the focus shifted from high-level observation to low-level kernel internals in xv6. Running xv6 on RISC-V inside QEMU made the boundary between user mode (U-mode) and supervisor mode (S-mode) concrete. I traced how a system call transitions across this boundary: from a user-level wrapper through the RISC-V `ecall` instruction, into the kernel trap vector, through the system call dispatch table, and down to the specific kernel handler. Finally, I applied this workflow by implementing a user-space `sleep` utility, linking it against the xv6 C library and build pipeline, and invoking the kernel's `pause` system call to suspend process execution.

Together, these laboratories provided a practical understanding of both the user-facing abstractions provided by an operating system and the low-level kernel mechanisms required to implement them.